

Research Paper

Optimizing potential-based reward automata in partially observable reinforcement learning using genetic local search[☆]Zhengwei Zhu^a, Zhixuan Chen^a, Chenyang Zhu^a^{*}, Wen Si^a, Fang Wang^b^a School of Computer Science and Artificial Intelligence, Changzhou University, China^b Department of Computer Science, Brunel University London, United Kingdom

ARTICLE INFO

Keywords:

Partially observable reinforcement learning
 Reward automata
 Heuristic algorithms
 Reward shaping

ABSTRACT

Partially observable reinforcement learning extends the reinforcement learning framework to environments in which agents have limited visibility of the state space, making it particularly relevant for applications in robotics and autonomous vehicle navigation. However, a primary challenge in partially observable reinforcement learning is defining effective reward functions that can guide the learning process despite partial observability. To address this challenge, this paper introduces a novel approach for constructing potential-based reward automata by employing genetic local search methods. Specifically, our method constructs these automata from compressed representations of exploration trajectories, which succinctly capture critical decision points and essential state transitions while eliminating redundant steps. By optimizing trajectory samples and shortening agent trajectories to their crucial transitions, our technique significantly reduces computational overhead. Formally, we define the learning objective as an optimization problem aimed at maximizing the log-likelihood of future observations while simultaneously minimizing the structural complexity of the learned reward automata. Furthermore, by incorporating value-based strategies to estimate potential values within the reward automata, our approach improves learning efficiency and facilitates the identification of optimal reward structures. We empirically evaluate our proposed method on seven partially observable grid-world benchmarks. Experimental results demonstrate that our method achieves superior performance relative to state-of-the-art reward automata-based techniques, exhibiting both accelerated learning speeds and higher accumulated rewards. Additionally, our genetic local search algorithm consistently outperforms comparative heuristic methods in terms of learning curves and reward accumulation.

1. Introduction

In many real-world scenarios, agents operate under conditions of partial observability, where access to the complete system state is restricted due to sensor limitations, occlusions, environmental uncertainty, or inherent task complexity. This constraint is prevalent across a broad range of applications, including autonomous driving, robotics, and human–robot interaction. For example, self-driving vehicles often contend with degraded visibility during adverse weather (Valiente et al., 2022; Levinson et al., 2011), aerial drones conducting search and rescue missions must navigate through unstructured environments with intermittent visual feedback (Curtis et al., 2023), and warehouse robots must make navigation decisions while avoiding dynamic and occluded obstacles (Lauri et al., 2022). In each case, the agent must reason

and act based on incomplete and noisy observations, highlighting the necessity of reinforcement learning algorithms designed specifically for partially observable environments.

Partially Observable Reinforcement Learning (PORL) extends the conventional RL framework to address decision-making under uncertainty by modeling the environment as a Partially Observable Markov Decision Process (POMDP). However, learning effective policies in POMDPs presents several challenges. First, due to the non-Markovian nature of the observation space, agents must infer latent states using historical information (Toro Icarte et al., 2023). This requires memory mechanisms that capture temporal dependencies beyond the current observation. Second, the design of reward functions becomes significantly more complex in the absence of full state visibility. Agents may mistakenly associate observations with rewards, leading to policy

[☆] This work was supported by National Natural Science Foundation of China (No. 62202067), CNPC Innovation Found (No. 2024DQ02-0501), Royal Society (IEC_NSFC_233444), Postgraduate Research and Practice Innovation Project of Jiangsu Province (No. KYCX24_3228) and Youth Science and Technology Talent Promotion Project of Jiangsu Province (No. JSTJ-2025-137).

^{*} Corresponding author.

E-mail address: zcy@cczu.edu.cn (C. Zhu).

<https://doi.org/10.1016/j.engappai.2026.114054>

Received 1 August 2024; Received in revised form 9 May 2025; Accepted 1 February 2026

Available online 9 February 2026

0952-1976/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

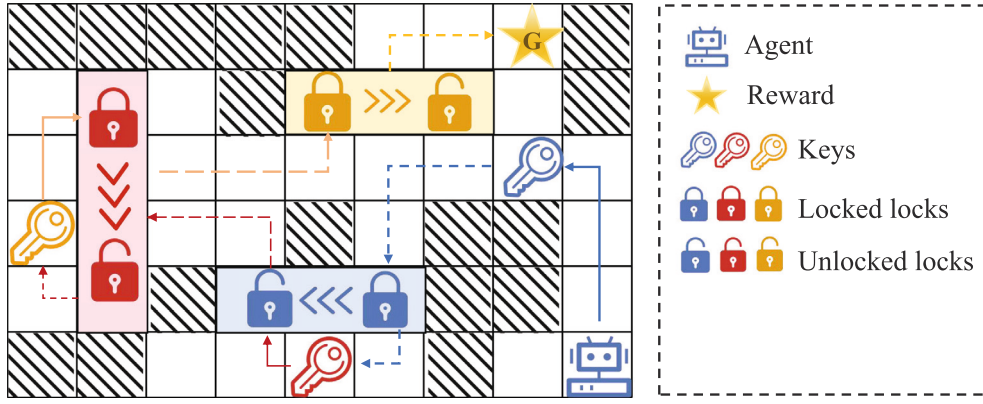


Fig. 1. A partially observable environment where the agent navigates through a sequence of rooms using keys to unlock corresponding locks. The agent can only observe the current room it occupies. It must collect and use keys (K_b , K_r , K_y) in the correct order to unlock locks (B_b , B_r , B_y) and reach the terminal room. After completing the task, the agent returns to the initial room M_a , and the environment resets. This setup illustrates the challenges of delayed rewards and limited observability in sequential decision-making tasks.

bias, especially in environments with delayed or sparse rewards. For instance, in a navigation task where an agent must sequentially collect keys to unlock rooms, the inability to observe the global structure of the environment makes it difficult to link current actions to future outcomes. As illustrated in Fig. 1, the agent must explore unknown rooms, use acquired keys in the correct order, and return to the starting point, all while reasoning over partial and delayed feedback.

To address these issues, we propose a novel PORL framework that learns Potential-Based Reward Automata (RA) directly from compressed exploration trajectories. The key idea is to extract decision-relevant information from trajectory data using a compression mechanism, thereby reducing memory usage while preserving essential state-action transitions. The resulting RA captures temporal dependencies in a structured form and eliminates the need for manually designed reward functions. To optimize the RA structure, we introduce a genetic local search algorithm that balances the expressiveness of the reward representation with the complexity of the automaton. We further incorporate value-based estimation techniques to assign potential values to RA states, guiding the agent toward optimal behavior under uncertainty. The learning process is formalized as maximizing the log-likelihood of future observations while penalizing automaton complexity to prevent overfitting.

The main contributions of this paper are summarized as follows:

- We propose a method for constructing Potential-Based Reward Automata from compressed trajectories, enabling automatic reward specification and temporal abstraction in partially observable environments.
- We design a genetic local search algorithm that optimizes RA structures with respect to both policy performance and model complexity. The algorithm facilitates efficient memory integration and task-specific reward modeling.
- We introduce a value-based potential estimation strategy that enhances the learning efficiency by shaping rewards within the RA framework, without requiring access to the full state space.
- We conduct comprehensive experiments across seven partially observable grid-world environments, demonstrating superior performance in terms of convergence speed, stability, and accumulated reward compared to existing RA-based methods. We also analyze the impact of adaptive ϵ -decay strategies for balancing exploration and exploitation under varying environmental complexities.

The structure of the paper is as follows. Section 2 describes the related work relevant to PORL and reward automatons addressing the reward functions in reinforcement learning. Section 3 introduces the preliminaries of the work. Section 4 describes our framework that con-

structs the RA from compressed trajectories with genetic local search. Section 5 evaluates our proposed approach with state-of-the-art approaches in seven different partially observable environments. We also demonstrate the advantages of genetic local search compared to other genetic-based algorithms. Lastly, we conclude our work in Section 6 and outline future work.

2. Related work

2.1. Partially observable reinforcement learning

Reinforcement learning, extensively studied in fully observable environments, is typically modeled using MDPs with various algorithms such as Q-learning, SARSA, and Policy Gradient (Jin et al., 2023; Zanette et al., 2020). However, in environments with incomplete information, RL is modeled as a POMDP (Laurent, 2008).

Planning and learning in POMDPs present significant computational complexity challenges. Computing the optimal policy is a PSPACE-complete problem even with known POMDP parameters (Papadimitriou and Tsitsiklis, 1987). Finding the optimal memoryless policy is still NP-hard (Vlassis et al., 2012). Model estimation for POMDPs is also complex; parameter estimation for certain Hidden Markov Models is at least as difficult as the parity problem for noise learning, implying that estimating POMDP parameters is equally challenging (Mossel and Roch, 2005).

Recent works have applied spectral methods to learn POMDPs, achieving results with polynomial sample complexity (Anandkumar et al., 2012). Jahromi et al. proposed an algorithm based on posterior sampling, assuming sufficient separability between models or successful estimation of belief states and transition kernels for efficient sampling (Jahromi et al., 2022). These assumptions simplify the estimation of model dynamics but limit the generality of the results. Jin et al. addressed the problem of exploration in incomplete POMDP learning, where the number of hidden states must be no greater than the number of observations (Jin et al., 2020). Their algorithm is specifically designed to exploit the incomplete structure of POMDPs, but it is unclear whether their technique can be extended to overcomplete settings. Golowich et al. recently developed the first quasi-polynomial time planning algorithm for a subclass of POMDPs under the γ -observability condition (Golowich et al., 2022). Their work focuses solely on computational efficiency and addresses only the planning problem, omitting estimation and exploration, which are crucial for POMDP learning.

Estimated states in POMDPs, representing the distribution of potential states, are typically modeled using recurrent neural networks and have proven effective for reinforcement learning (Igl et al., 2018). However, POMDPs can lead to degenerate states that disregard historical

observations (Gangwani et al., 2020). Our approach utilizes RA as a historical memory tool to learn strategies directly from experience.

Building on this foundation, recent advances in memory-augmented RL have introduced benchmark suites and architectural innovations aimed at effectively capturing long-term dependencies in partially observable environments. POPGym offers a diverse collection of POMDP tasks specifically designed for evaluating memory mechanisms (Morad et al., 2023). Similarly, MemoryGym provides a systematic framework for benchmarking memory-based agents across a wide range of task domains and temporal complexities (Pleines et al., 2023). In parallel, transformer-based sequence models have demonstrated superior performance over recurrent neural networks in several POMDP benchmarks, highlighting their enhanced capacity for handling extended temporal credit assignment and partial observability (Lu et al., 2024).

2.2. Reward automatas based reinforcement learning

Research has employed linear temporal logic (LTL) and related formalisms to guide RL in MDPs (Lacerda et al., 2014; Hasanbeig et al., 2018). Traditional methods often convert LTL specifications into finite state machines, which then reward agents upon reaching designated acceptance states using a cross-product baseline learning strategy. An alternative approach, eschewing automata, harnesses the structural properties of LTL to enhance learning efficiency by decomposing complex LTL tasks into simpler sub-tasks. These sub-tasks are tackled in parallel via off-policy learning (Icarte et al., 2018).

Icarte et al. have advanced this field by introducing Reward Automatas and developing the QRM method, which outperforms both the cross-product baseline and hierarchical reinforcement learning approaches in efficiency (Icarte et al., 2018). They further adapted these methods for use in partially observable environments (Toro Icarte et al., 2023). RAs have since been employed across various domains, including planning, robotics, multi-agent systems, and lifelong learning, to manage partial observability (Illanes et al., 2020; Camacho et al., 2021; Neary et al., 2020; Zheng et al., 2022; Icarte et al., 2019). Innovations have also been made in learning RAs directly from empirical data (Furelos-Blanco et al., 2021; Dohmen et al., 2022). Our study introduces a novel RA learning mechanism that incorporates reward constraints, significantly enhancing practical and theoretical understanding. Icarte et al. have utilized implementations such as forbidden search and local search to optimize learning of objective functions (Icarte et al., 2019; Toro Icarte et al., 2023). In this work, we integrate heuristic algorithms with local search to develop a genetic local search method, refining the optimization of effective solutions.

Concurrently, recent studies have expanded the capabilities and resilience of reward automata. Parac et al. proposed PROB-IRM, a framework based on inductive logic programming for learning robust reward machines from noisy behavioral traces (Paracá et al., 2024). Levina et al. introduced Numeric Reward Machines, an extension that incorporates continuous-valued observations into the automaton structure to handle numeric decision tasks (Levina et al., 2024). Bourel et al. analyzed low-regret exploration strategies within RA-augmented environments and provided theoretical guarantees on policy performance (Bourel et al., 2023). In addition, Toro et al. developed contextual pre-planning methods that utilize RA abstractions to guide structured deep RL training (Toro-Rey et al., 2023). Addressing multi-agent coordination under temporal logic constraints, Zhu et al. introduced a synchronized decentralized training paradigm for multi-agent reinforcement learning. Their approach synthesizes decomposed reward machines from GR(1)-based specifications and incorporates the resulting strategies as prior knowledge, effectively mitigating non-Markovian dependencies while enhancing learning stability and coordination efficiency (Zhu et al., 2024).

Our methodology leverages RAs and a genetic local search strategy to address objective functions and enhance solution efficacy, contrasting sharply with traditional approaches in POMDPs. Traditional

POMDP techniques are plagued by significant computational hurdles, including PSPACE-completeness for optimal policy computation and NP-hardness for optimizing memoryless policies. Recent advancements involve spectral methods and posterior sampling for learning POMDPs, yet these often presume strong reachability or model separability, constraining their applicability. Moreover, while some algorithms explore POMDPs, they generally neglect crucial aspects like estimation and systematic exploration. Unlike these traditional methods, our approach integrates reward constraints directly within the learning process and applies a hybrid of heuristic algorithms and local search. This strategy provides a flexible and potent solution for navigating the complexities of partially observable environments. Not only does it improve the efficiency of learning objective functions, but it also refines existing solutions, thereby surmounting traditional limitations and demonstrating enhanced adaptability and effectiveness in complex settings.

3. Preliminaries

Reinforcement learning agents learn strategies from experience. In partially observable environments, the underlying model is typically assumed to be a POMDP, as defined in Definition 3.1.

Definition 3.1 (POMDP). A standard Partially Observable Markov Decision Process is defined as a tuple $P_o = (S, O, A, r, p, \omega, \gamma, \mu)$, where:

- S is a finite set of states, where each state $s \in S$ represents a possible situation in the environment.
- O is a finite set of observations, where each observation $o \in O$ represents a possible outcome perceived by the agent in a given state.
- A is a finite set of actions, where each action $a \in A$ represents a choice available to the agent in any state $s \in S$.
- $r : S \times A \rightarrow \mathbb{R}$ is the reward function, where $r(s, a)$ denotes the reward received by the agent for taking action a in state s .
- $p : S \times A \times S \rightarrow [0, 1]$ is the state transition probability distribution, defined as Eq. (1):

$$p(s, a, s') = p(s_{t+1} = s' \mid s_t = s, a_t = a), \sum_{s' \in S} p(s, a, s') = 1 \quad (1)$$

where $p(s_{t+1} = s' \mid s_t = s, a_t = a)$ is the conditional probability of transitioning to state s' from state s when action a is taken.

- $\omega : O \times S \rightarrow [0, 1]$ is the observation probability distribution, defined as Eq. (2):

$$\omega(o \mid s) = \omega(o_t = o \mid s_t = s), \sum_{o \in O} \omega(o \mid s) = 1 \quad (2)$$

where $\omega(o_t = o \mid s_t = s)$ is the conditional probability of observing $o \in O$ given state $s \in S$.

- $\gamma \in [0, 1)$ is the discount factor, which discounts the value of future rewards. A smaller γ indicates a preference for short-term rewards, while a larger γ indicates a preference for long-term rewards.
- $\mu \in \Delta(S)$ is the initial state distribution, where $\mu(s_0) = p(s_0 = s)$ is the probability that the agent starts in state $s_0 \in S$.

The environment begins in an initial state $s_0 \sim \mu$. At time step t , the agent is in state $s_t \in S$, performs action $a_t \in A$, receives an immediate reward $r_{t+1} = r(s_t, a_t, s_{t+1})$, and transitions to state s_{t+1} according to the transition probability $p(s_{t+1} \mid s_t, a_t)$. However, the agent does not directly observe s_t . Instead, the agent observes $o_t \in O$, where $\omega(o_t \mid s_t)$ is the probability of observing o_t given state s_t .

Q-learning is a reinforcement learning algorithm based on value iteration, with the primary objective of learning a Q-function $Q(s, a)$ to estimate the expected cumulative reward obtained after taking action a in state s . The Q-learning algorithm updates the Q-value using the following rule, shown in Eq. (3):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)] \quad (3)$$

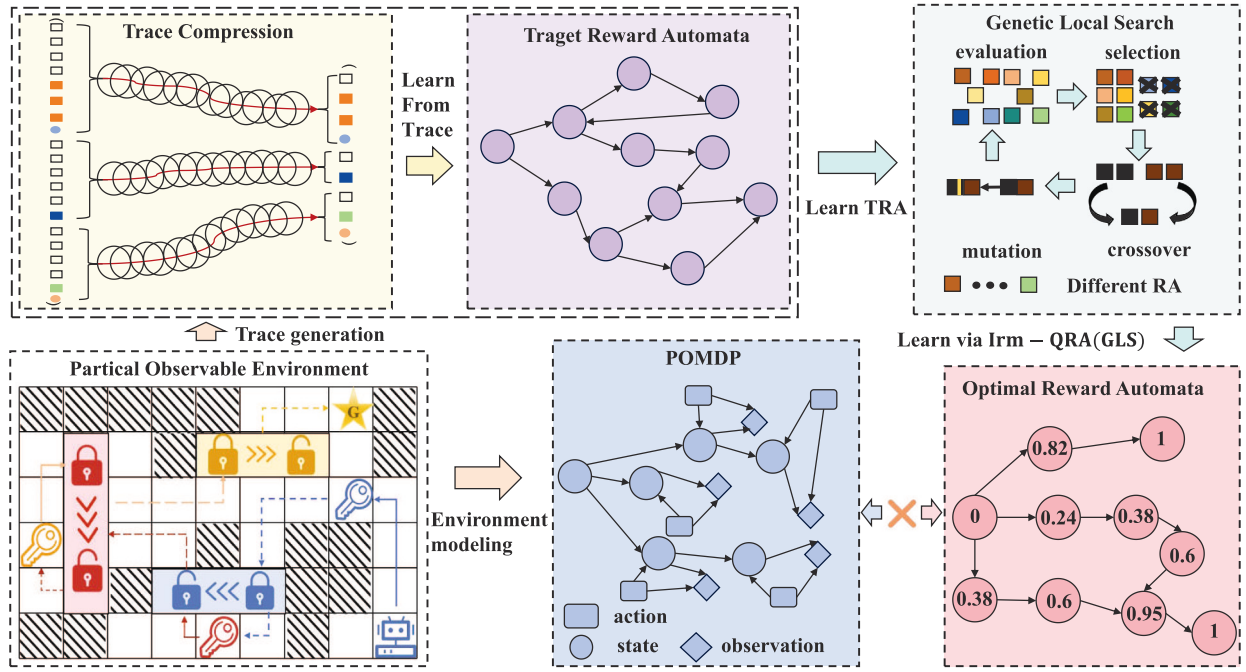


Fig. 2. Overall framework of learning reward automata using genetic local search.

where α is the learning rate, controlling the step size of the Q-value update, γ is the discount factor, balancing the importance of current and future rewards, r_t is the immediate reward obtained after performing action a_t in state s_t , and $\max_{a'} Q(s_{t+1}, a')$ is the maximum Q-value for the next state s_{t+1} over all possible actions a' . Through interaction with the environment, the agent continuously updates the Q-values and eventually learns the optimal policy.

4. Learning reward automata using genetic local search

This paper introduces a method for learning potential-based reward automata in partially observable environments by leveraging a genetic local search strategy applied to compressed agent trajectories. The overall framework is shown in Fig. 2. The core idea is to compress exploration trajectories to extract essential transitions and decision points, which significantly reduces computational overhead while preserving crucial behavioral patterns.

We formulate the RA construction as an optimization problem that simultaneously maximizes the likelihood of observed trajectories and minimizes the structural complexity of the RA. A genetic local search algorithm is employed to efficiently explore the space of candidate automata and select the one that best balances predictive fidelity and interpretability.

After obtaining the optimized RA, we apply value iteration to compute the associated potential values. These values are then used for potential-based reward shaping, where the reward function is modified based on the difference in potential between successive automaton states. This mechanism links the POMDP with the RA via a potential-based reward function, improving the alignment between long-term objectives and local decision-making.

By enriching the reward signal with structured temporal information from the RA, our approach enhances policy learning under partial observability, ultimately supporting the synthesis of more effective and sample-efficient strategies.

4.1. POMDP with potential-based reward automata

To find a RA that can act as external memory for solving partially observable problems, our approach accepts as input a set of high-level

propositional symbols Σ and a labeling function L capable of detecting these symbols. The agent needs to learn RA that can remember sufficient information to make accurate Markov predictions for the next observation. Inspired by work in Toro Icarte et al. (2023), we define an optimal RA that uses its state U and state transition function δ_u to track relevant past information, making the partially observable environment P_o Markovian in terms of $O \times U$.

In partially observable environments, a trajectory typically describes the sequence of states, actions, and rewards experienced by an agent over a period of time. The formal definition is shown in Definition 4.1.

Definition 4.1 (Trajectory in Partially Observable Environments). A trajectory is a sequential record of an agent's interactions with its environment, capturing states, actions, and rewards. In partially observable environments the agent does not have access to the complete state information. Instead, it receives observations o_t , which provide incomplete or noisy information about the underlying true state. Formally, a trajectory is represented as:

$$T = \{(o_0, a_0, r_0), (o_1, a_1, r_1), \dots, (o_t, a_t, r_t)\},$$

where, at each time step t :

- o_t denotes the observation available to the agent at time step t , based on which the agent selects action.
- a_t is the action executed by the agent,
- r_t is the reward received from the environment.

Consider an agent navigating a maze under partial observability. A trajectory in such an environment can be represented as $\{(o_0, \text{right}, 0), (o_1, \text{up}, 0), (o_2, \text{up}, 0), (o_3, \text{goal}, 1)\}$. In this example, the agent perceives four observations (o_0, o_1, o_2, o_3) , executes three actions ("right", "up", "up"), and receives rewards at each time step, culminating in a terminal reward upon reaching the goal. This trajectory captures the temporal sequence of observations, actions, and rewards, offering a detailed record of the agent's interaction with the environment.

Such trajectory representations are crucial for understanding and improving the agent's behavior, particularly in POMDPs, where the full environment state is not directly accessible. By analyzing these

sequences, one can infer structured patterns that are essential for robust policy learning.

To formalize this, we define an optimal RA derived from trajectories, as introduced in Definition 4.2. Given a trajectory generated by a policy in the partially observable process P_o , represented as $(o_0, a_0, o_1, a_1, \dots, o_t, a_t)$, we recursively define auxiliary automaton states using Eq. (4).

$$x_0 = u_0, \quad x_t = \delta_u(x_{t-1}, L(o_{t-1}, a_{t-1}, o_t)) \quad (4)$$

where u_0 is the initial RA state, δ_u is the RA transition function, and $L(o_{t-1}, a_{t-1}, o_t)$ denotes the label assigned to the transition induced by observation-action pairs. This construction provides a mechanism to align the trajectory with an underlying RA structure, enabling reward shaping and policy guidance in partially observable settings. Given these definitions, the conditional probability of observing the next observation o_{t+1} and receiving the reward r_t given the history of interactions up to timestep t , represented as trajectory T_t , satisfies the Markov property as shown in Eq. (5):

$$Pr(o_{t+1}, r_t | T_t) = Pr(o_{t+1}, r_t | o_t, x_t, a_t) \quad (5)$$

This equation indicates that the probability of the next observation and reward depends exclusively on the current observation o_t , auxiliary state x_t , and action a_t .

Definition 4.2 (Optimal RA from Trajectories). An optimal RA is defined as a tuple $R_P = \langle U, \phi, e_0, \delta_u, \delta_r, x_{i,t}, N_{u,\sigma} \rangle$, where:

- U is a finite set of states, and $e_0 \in U$ is the initial state.
- ϕ is the value of the potential function.
- $\delta_u \in U \times 2^\Sigma \rightarrow U$ is the state transition function.
- $\delta_r \in U \times 2^\Sigma \rightarrow \mathbb{R}$ is the reward function.
- The auxiliary variable $x_{i,t} \in U$ denotes the state of the RA in the observation trajectory T_i until after time t , and $x_{i,t+1} = \delta_u(x_{i,t}, L(e_{i,t+1}))$.
- The variable $N_{u,\sigma} \subseteq 2^\Sigma$ is the set of all the next high-level observations σ emerging from the RA state u and the high-level observation σ in the trajectory T , it satisfies $\forall u \in U, \sigma \in \Sigma, L(e_{i,t+1}) \in N_{x_{i,t}, L(e_{i,t})}$.

Our approach leverages potential-based reward shaping (Zhu et al., 2022) through the construction of the extended process P_R , as formalized in Definition 4.3. This method augments the original POMDP without modifying its set of optimal strategies. Specifically, it introduces a potential function ϕ over the automaton states, which reshapes the reward function by incorporating differences in potential across transitions. Since potential-based shaping preserves policy optimality, identifying a suitable ϕ can accelerate the learning process while ensuring that the learned strategies remain optimal with respect to the original reward structure.

Definition 4.3 (POMDP with Potential-based Reward Automata). Given a POMDP P_0 and an optimal RA R_P , where P_0 and R_P share the labeling function $\mathcal{L}$, a POMDP with potential-based RA is defined as:

$$P_R = (\hat{S}, O, \hat{u}_0, A, \Psi, \Gamma, \hat{r}, \hat{\phi}, \omega, \gamma, \mu),$$

where:

- $\hat{S} = S \times U$, the combined state space of the original POMDP states and the states of the RA.
- $\hat{u}_0 = s_0 \times u_0$, the initial state composed of the initial state of the POMDP and the initial state of the RA.
- $\Psi : \hat{S} \rightarrow [0, 1]$ is the potential function defined for the reward structure.
- $\Gamma : O \times A \rightarrow \Sigma$ is the labeling function mapping POMDP transitions to the automaton symbols.

- The reward function $\hat{r}$ is defined as Eq. (6):

$$\hat{r}(\langle s, e \rangle, a, \langle s', e' \rangle) = \delta_r(e, \mathcal{L}(s, a, s')) + \begin{cases} 0 & \text{if } e = e' \\ \gamma\phi(e') - \phi(e) & \text{otherwise} \end{cases} \quad (6)$$

- $\hat{p}(\langle s', e' \rangle | \langle s, e \rangle, a) = p(s' | s, a)$, the state transition probability.

In P_R , an agent executes an action a to transition from state s to state s' , with the transition labeled by a function $\mathcal{L} : S \times A \times S \rightarrow 2^\Sigma$. When the pair $\langle e, \mathcal{L}(s, a, s') \rangle$ belongs to the domain of the transition function δ_e of a reward automaton R_P , the automaton transitions from state e to $\delta_e(e, \mathcal{L}(s, a, s'))$. If the label does not activate any transition, the automaton remains in its current state. Assume that both the state space S and action space A are finite. The immediate reward function $r(s, a)$ is uniformly bounded by a constant $R_{\max}$, satisfying $|r(s, a)| \leq R_{\max}$ for all $(s, a) \in S \times A$. The policy $\pi(a | s, \theta)$ is parameterized and continuously differentiable with respect to θ , and the discount factor γ satisfies $0 < \gamma < 1$. We then use Theorem 4.4 to prove that policy optimization in the extended model P_R leads to strategies that remain optimal in the original POMDP P_o , thereby ensuring theoretical soundness and practical reliability of the proposed reward shaping framework.

Theorem 4.4. The optimal policy of the P_R is identical to the optimal policy of the original POMDP P_0 given ϕ is bounded.

Proof. Fix an arbitrary policy π . The value function under the original POMDP P_0 is defined in Eq. (7):

$$V_{P_0}^\pi(s) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, s_{t+1}) \mid s_0 = s \right] \quad (7)$$

The value function in the extended model P_R is given by Eq. (8):

$$V_{P_R}^\pi(\langle s, e \rangle) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \hat{r}(\langle s_t, e_t \rangle, a_t, \langle s_{t+1}, e_{t+1} \rangle) \mid \langle s_0, e_0 \rangle = \langle s, e \rangle \right] \quad (8)$$

Substituting the shaped reward definition into Eq. (8) yields Equation (9):

$$V_{P_R}^\pi(\langle s, e \rangle) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t (\delta_r(e_t, \mathcal{L}(s_t, a_t, s_{t+1})) + \gamma\phi(e_{t+1}) - \phi(e_t)) \mid \langle s_0, e_0 \rangle = \langle s, e \rangle \right] \quad (9)$$

$$= \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \delta_r(e_t, \mathcal{L}(s_t, a_t, s_{t+1})) \mid \langle s_0, e_0 \rangle = \langle s, e \rangle \right] + \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^{t+1} \phi(e_{t+1}) - \gamma^t \phi(e_t) \right] \quad (10)$$

where the second term is a telescoping sum. As shown in Eq. (11), this sum simplifies to:

$$\sum_{t=0}^T (\gamma^{t+1} \phi(e_{t+1}) - \gamma^t \phi(e_t)) = -\phi(e_0) + \gamma^{T+1} \phi(e_{T+1}) \quad (11)$$

Since ϕ is bounded and $0 < \gamma < 1$, the term $\gamma^{T+1} \phi(e_{T+1})$ converges to 0 as $T \rightarrow \infty$. Thus, we obtain Eq. (12):

$$\mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^{t+1} \phi(e_{t+1}) - \gamma^t \phi(e_t) \right] = -\phi(e) \quad (12)$$

Substituting Eq. (12) into Eq. (10), we derive the relationship in Eq. (13):

$$V_{P_R}^\pi(\langle s, e \rangle) = V_{P_0}^\pi(s) - \phi(e) \quad (13)$$

Since $\phi(e)$ is independent of future actions, it does not affect the relative ordering of policy values. Therefore, for any two policies π_1 and π_2 , Eq. (14) holds:

$$V_{P_R}^{\pi_1}(\langle s, e \rangle) \geq V_{P_R}^{\pi_2}(\langle s, e \rangle) \iff V_{P_0}^{\pi_1}(s) \geq V_{P_0}^{\pi_2}(s) \quad (14)$$

Thus, the optimal policies for P_0 and P_R are identical, completing the proof. $\square$

4.2. Learning optimal reward automata from trajectories

Our method involves learning RA from the trajectories generated by the agent’s interactions with the environment. Initially, we compress the trajectories to optimize trajectory samples and reduce computational costs. We define the learning objective function as an optimization problem, aiming to maximize the log-likelihood of future observations while minimizing the complexity of the RA. Additionally, we incorporate reward shaping techniques to guide the agent in learning optimal RA efficiently.

The Target Reward Automata (TRA) operates based on the high-level observation σ_t given by the labeling function $L(e_t)$, where e_t denotes the experience at time t . The first step is to transform each trajectory T_i from a sequence of interactions with the environment to a sequence of truth-value assignments to $\mathbb{P}$ via L , as shown in Eq. (15).

$$T_i = (o_{i,0}, a_{i,0}, r_{i,0}, \dots, o_{i,t_i}) \Rightarrow \tau_i = (\sigma_{i,0}, \dots, \sigma_{i,t_i}) \quad (15)$$

In the abstract space defined by $\mathbb{P}$ and L , the same high-level observation often occurs several times in succession. For example, a typical high-level trajectory in the motivating example shown in Fig. 1 is shown below:

$$\tau = (\square, \square, \square, \square, \blacksquare, \blacksquare, \square, \square, \square, \square, \blacksquare, \blacksquare, \square, \square, \square, \blacksquare, \blacksquare, \bullet)$$

After compression, the trajectory takes the following form:

$$\tau_c = (\square, \square, \blacksquare, \square, \blacksquare, \square, \blacksquare \bullet)$$

In the trajectories we present, $\square$ indicates that the agent is in an empty room or corridor. $\square$, $\square$, and $\square$ represent the agent exploring after acquiring keys K_b , K_r , and K_y respectively, without having unlocked and entered the corresponding rooms. Meanwhile, $\blacksquare$, $\blacksquare$, and $\blacksquare$ signify the agent unlocking the corresponding locks M_b , M_r , and M_y with the acquired keys and then being inside those rooms. Finally, $\bullet$ represents the agent reaching the goal.

Compressing the trajectory is an optional preprocessing step that offers both advantages and disadvantages. The advantage is that, with a fixed computational budget, it significantly improves the quality of the RAs found by the model and reduces the computational cost of traversing the trajectory. The disadvantage is that compression may result in a loss of information if consecutive observations provide more insight into the current POMDP state. While compression may help the model find high-quality RAs faster, it might not find the optimal solution.

Our goal is to find a RA R_p that can predict $L(e_{i,t+1})$ from $L(e_{i,t})$ based on the current RA state $x_{i,t}$, where $e_{i,t+1}$ is the empirical $(o_{i,t}, a_{i,t}, o_{i,t+1})$ and $e_{i,0}$ is defined as $(\Phi, \Phi, o_{i,0})$. The model parameters include the set of trajectories T , the set of propositional symbols $\mathbb{P}$, the labeling function L , and the maximum number of states $u_{\max}$ of the RA. The model also uses the set $I = \{0, \dots, n\}$, the set $T_i = \{0, \dots, t_{i-1}\}$, and Σ , which contains all the high-level observations in T .

The objective function of the TRA, shown in Eq. (16), is defined to maximize the log-likelihood of predicting future observations and minimize the complexity of the RA. This ensures that the learned RAs effectively characterize the environment and can be used as memory in the RL algorithm. The objective function uses state transition probabilities δ_u to predict the likelihood of the next observation, computed using the current state and transfer function of the RA given the current state and action. It also considers the number of states and transitions in the RA to ensure simplicity and efficiency.

$$\begin{aligned} & \arg \min_{(U, \phi, u_0, \delta_u, \delta_r, x_{i,t+1}, N_{u,\sigma})} \sum_{i \in I} \sum_{t \in T_i} \log N_{x_{i,t}, L(e_{i,t})} + \lambda \|U\|^2 \\ & s.t. (U, \phi, u_0, \delta_u, \delta_r) \in R_p, |U| \leq u_{\max} \end{aligned} \quad (16)$$

Here, λ is a regularization parameter introduced to penalize the complexity of the RA in the objective function U . The parameter λ regulates the trade-off between fitting the observed trajectory data and maintaining the structural simplicity of the automaton. To select an appropriate value for λ , we construct a validation set by sampling representative trajectories from the experimental environments and conduct a grid search. The value of λ that maximizes the penalized objective is retained. A higher λ encourages more compact automata, reducing model complexity and the risk of spurious transitions, whereas a lower λ permits more expressive automata structures at the expense of potentially overfitting to training data.

To ensure the generation of valid RA, we impose constraints that limit the number of states to a predefined upper bound $u_{\max}$. Each state $x_{i,t}$ represents the RA state at time step t , initialized at u_0 and updated according to the state transition function δ_{iu} . The set $N_{u,\sigma}$ contains all labels $L(e_{i,t+1})$ observed when the automaton transitions from state u upon receiving symbol σ . The learning objective is to maximize the entropy of the distribution defined over $N_{u,\sigma}$, thereby promoting uniform predictions across potential transitions and preventing overly deterministic or trivial behavior. Crucially, this formulation guarantees that as the number of training trajectories approaches infinity, any RA optimized under this objective converges to an optimal automaton solution for the TRA problem, ensuring theoretical soundness and asymptotic consistency given sufficient data coverage.

However, directly solving this optimization problem becomes computationally infeasible as the problem scales, prompting the use of heuristic algorithms. Heuristic methods provide polynomial-time approximate solutions to NP-hard problems, typically emphasizing practical effectiveness over theoretical optimality. To address this challenge, we propose a genetic local search algorithm that leverages trajectory sets as populations. Within each generation, candidate RAs undergo fitness evaluations based on their performance within the task domain, followed by local search refinements. This approach systematically balances global exploration through genetic operations with targeted local improvements, thus efficiently identifying high-quality solutions in complex problem spaces.

Algorithm 1 Solving TRA Problems Using Genetic Local Search Methods

```

1: Input: Observation Set  $\Sigma$ , trajectory  $T$ ,  $u_{\max}$ ,  $t_{\max}$ ,  $R$ ,  $n$ 
2: Output:  $R^*$ 
3: Initialize  $t \leftarrow 0$ ,  $c^* \leftarrow \infty$ ,  $R^* \leftarrow \text{None}$ ,  $G \leftarrow \text{empty}$ 
4: while  $t \leq t_{\max}$  do
5:    $c_p \leftarrow \infty$ ,  $c \leftarrow \text{evalra}(R, T)$  using Equation (17)
6:   if  $c < c^*$  then
7:      $c^* \leftarrow c$ ,  $R^* \leftarrow R$ ,  $(b, G^*, f) \leftarrow \text{create}(G)$ 
8:   end if
9:   while each  $g \in ngen$  do
10:     $f \leftarrow \text{map}(G^*)$  Obtain the transformed population and fitness
    function (19) using Equation (20)
11:    select  $G$  from  $(G^*)$  with Equation (21)
12:  end while
13:  for  $t \leq t_{\max}$  do
14:     $t \leftarrow t + 1$ ,  $c_p \leftarrow c$ ,  $\kappa \leftarrow \text{get.neighborhood}(R, \Sigma, u_{\max})$ 
15:    for  $R_n \in \kappa$  do
16:       $c_n \leftarrow \text{evara}(R_n, G)$ 
17:    end for
18:    if  $c_n < c$  then
19:       $c \leftarrow c_n$ ,  $R \leftarrow R_n$ ,  $(G^*, f) \leftarrow \text{map}(G)$ 
20:    end if
21:  end for
22: end while

```

Algorithm 1 illustrates the process of learning RA using genetic local search. We define the neighborhood of a RA as those RA that differ by only one transition path, namely, by removing, adding, or modifying a

transition. These automata are evaluated using the objective function of the target RA. This algorithm treats trajectories as populations and formalizes the process of learning RA using genetic operations on populations. Each iteration involves genetic operations on RA, including selection, crossover, and mutation, based on evaluated strengths and weaknesses, which correspond to fitness.

The algorithm receives the observation set Σ , the training trajectory T , the maximum number of RA states $u_{\max}$, and termination criteria such as the time limit or the maximum number of steps $t_{\max}$. It also receives a randomly generated initial RA R and the number of iterations n . Auxiliary variables are defined to aid in learning the RA. In each loop, optimization of the RA using a variant of the genetic algorithm, creating a population, and generating the next generation using crossover and mutation operations. Each individual uses the TRA objective function to calculate the fitness value by evaluating the function evalra.

The method evalra(R, T) evaluates the performance of a given rule set R on a trajectory T , returning an evaluation cost c shown in Eq. (17).

$$c = \sum_{i \in E} \log(\text{len}(F_{o,i})) \quad (17)$$

Through the create module, users can inherit existing classes and add additional attributes or methods to tailor them to specific optimization problems. The individual with the highest fitness value is selected as the next-generation population. The fitness of individuals in the current population is shown in Eq. (18).

$$f(G^*) = (f_1(g_1), f_2(g_2), \dots, f_n(g_n)) \quad (18)$$

Here g_i are individuals in the population. If normalization is required, the fitness function is improved as in Eq. (19),

$$f_{\text{norm}}(G^*) = \frac{f(G^*) - \min(f)}{\max(f) - \min(f)} \quad (19)$$

In this process, we use the map function to transform the population G^* to adapt to the next genetic operation, returning the transformed population and the fitness function f . Amdahl's Law limits the achievable speedup S by considering the fraction of the task that remains serial, as shown in Eq. (20).

$$S = \frac{1}{\eta + \frac{1-\eta}{\rho}} \quad (20)$$

Here ρ is the number of processing units, and η is the fraction of the task that remains serial. The selection probability is shown in Eq. (21).

$$\kappa = \frac{f_i}{\sum_{j \in G^*} f_j} \quad (21)$$

Here f_i is the individual i 's fitness value, and $\sum_{j \in G^*} f_j$ is the sum of all individual fitness values in the tournament. Using the get.neighborhood function to obtain the neighborhood, the cost of the RA within it is evaluated. If the cost is low, the best cost and the best RA are updated. When the number of steps reaches the maximum specified, the loop stops. Finally, the best RA is returned.

The core objective of Algorithm 1 is to identify an optimal RA through an efficient genetic local search approach. Initially, the algorithm evaluates each candidate RA based on its capability to accurately represent the observed transformation patterns while maintaining low structural complexity. Following this evaluation step, genetic operations are executed to progressively refine the RA, as depicted in Fig. 3. Specifically, these operations begin with a fitness-based selection mechanism, retaining candidate RAs that demonstrate superior performance according to predefined evaluation criteria, thereby discarding less promising candidates. Subsequently, crossover operations are probabilistically applied to combine advantageous structural characteristics of selected RAs, producing new offspring that integrate beneficial attributes inherited from their parent automata. To ensure sufficient

diversity and facilitate exploration of alternative RA structures, mutation operations are also applied probabilistically, introducing incremental variations into the offspring automata. Collectively, these genetic procedures provide robust global search capabilities, which are further complemented by a local optimization phase. In this local search, the algorithm systematically explores RA neighborhoods, where each neighbor is generated by altering precisely one transition of the automaton, thus enabling fine-grained optimization and convergence toward the optimal solution.

4.3. Value-based potential value calculation

We then use the value iteration strategy to compute potential functions over RA states. The core idea is to treat the RA as a POMDP that approximates the expected discounted reward for each RA state. By assigning a potential value to each RA state, we define a reward function that incentivizes the agent to progress towards task completion. This approach is applicable only to simple RA, as it does not incorporate information about the environment state S and action $a \in A$.

Intuitively, this can be viewed as a POMDP where each state transition in the RA corresponds to a deterministic action. Methods such as value iteration can be used to compute the value of each state $v_j(u) = \max q_j(u, \sigma)$ under the optimal policy. This computation is performed given $U, F, P, \delta_u, \delta_r$, and γ , as shown in Algorithm 2.

Algorithm 2 Value-iteration Based Reward Shaping Algorithm

```

1: Input: State list  $U$ , State transition function  $\delta_u$ , State bonus value  $U_r$ , Terminal state term $_u$ , Reward Shaping Discount Factors  $\gamma_s$ 
2: Output: State value estimation  $v^*$ 
3: Initialize  $V \leftarrow \text{dict}[(u_i, 0) \text{ for } u_i \text{ in } U]$ ,  $V_{\text{error}} \leftarrow \infty$ 
4: Initialize  $E_{\min} \leftarrow 10^{-4}$ 
5: while  $V_{\text{error}} > E_{\min}$  do
6:    $V_e \leftarrow 0$ ,  $u \leftarrow U - \text{term}_u$ 
7:   for  $u_i$  in  $\text{term}_u$  do
8:      $u \leftarrow u \cup \{u_i\}$ ,  $q_u \leftarrow []$ 
9:     for  $u_j$  in  $u$  do
10:      for  $u_k$  in  $\delta_u$  do
11:         $r \leftarrow U_r[u_j].\text{get\_reward}(o)$ 
12:         $q_u.\text{append}(r + \gamma_s \times V[u_i])$ 
13:      end for
14:       $v_{\text{new}} \leftarrow \max(q_u)$ 
15:       $V_{\text{error}} \leftarrow \max(V_{\text{error}}, |v_{\text{new}} - V[u_k]|)$ 
16:       $V[u_j] \leftarrow v_{\text{new}}$ 
17:    end for
18:     $u \leftarrow u - \{u_i\}$ 
19:  end for
20: end while

```

The computed state value estimate v is guaranteed to converge to v^* due to the well-established convergence properties of value iteration. Once v^* is computed, we define the potential function as $\phi(s, u) = -v_j(u)$ for each environment state s and RA state u . This negative feedback mechanism guides the agent towards the RA state that corresponds to task completion.

5. Experiments and results

5.1. Experimental setup

In this section, we empirically evaluate our approach across seven partially observable environments. First, we compare the learning performance of a combination of Genetic Local Search with Potential-based Reward Automata (GLS-PRA) and policies using double DQN (Icarte et al., 2018) and deep QRM (Icarte et al., 2018). Additionally, we assess the effectiveness of Genetic Local Search (GLS), the Local Search

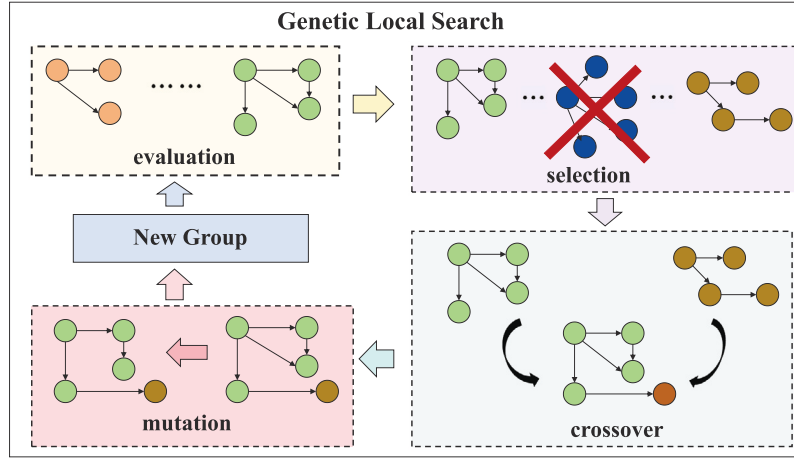


Fig. 3. Illustration of the genetic local search process for optimizing reward automata. The procedure includes evaluation, fitness-based selection, crossover, and mutation operations to evolve candidate automata. These global search mechanisms are followed by a local search phase that iteratively refines the automata by modifying single transitions, facilitating convergence to an optimal structure.

Table 1

The reward automata-based algorithms for comparison.

Algorithm	Experiential learning	Reward shaping	Heuristic
DDQN (Icarte et al., 2018)	×	×	×
LRM+DDQN (Icarte et al., 2019)	✓	×	Tabu Search
LRM+DQRM (Icarte et al., 2019)	✓	×	Tabu Search
LRM+DDQN(LS) (Toro Icarte et al., 2022)	✓	×	Local Search
LRM+DQRM(LS) (Toro Icarte et al., 2022)	✓	×	Local Search
GLS-PRA	✓	✓	Genetic Local Search

Table 2

Hyper-parameter settings for the experiments.

Hyper-parameter	Definition	Value
α	Learning rate	5×10^{-5}
γ	Discount factor for Q-learning algorithm	0.9
γ_g	Discount factor for value iteration in reward shaping	0.9
ϵ	Parameter for ϵ -greedy action selection	0.1, 0.09, 0.08
u_{max}	Maximum allowed uncertainty	10
$total_steps$	Number of total steps	4×10^6
$buffer_size$	Experience replay buffer size	10 000
ini_steps	Number of steps to initialize learning and sampling	20 000
$episode$	Number of steps per episode	5000, 10 000

Algorithm with Restart (LS) (Toro Icarte et al., 2022), and Tabu Search (TS) (Icarte et al., 2019) in solving the TRA problem. Table 1 presents the RA-based algorithms used for comparison. Finally, we investigate the impact of reward shaping, as well as the effects of randomness and the exploration–exploitation trade-off, on algorithm training results in more complex environments.

We tested three versions of the Learning Reward Automata method: LRM+DDQN (Icarte et al., 2018), LRM+DQRM (Icarte et al., 2018), and our method GLS-PRA. All methods learn reward automatons through experience, but LRM+DDQN uses Double Deep Q-Network (DDQN) (Icarte et al., 2018) to learn the policy, whereas LRM+DQRM uses a modified version of Deep Q-Learning with Reward Automatons (DQRM) (Icarte et al., 2018). Our method, GLS-PRA, employs Q-learning to learn RA while incorporating reward shaping to guide the learning process. To learn RA, these methods solve the Target Reward Automata using GLS, LS, or TS. The learning parameters are set as shown in Table 2.

For all methods, the neural networks used have five fully connected layers with 64 neurons per layer. At each step, we train the network using 32 samples from a replay buffer of size 100,000, with a learning rate of 5×10^{-5} . The target network is updated every 100 steps. Each

method is executed on a 13th Gen Intel® Core™ i7-13700 x24 processor with 16 cores and 1TB of RAM.

We conducted experiments across seven environments, classified into two categories: **Discrete Logical Environments** and the **Dynamic Navigation Environment**. The first category comprises six structured grid worlds: Cookie Domain, Keys Domain, Symbol Domain, Symbol Matching Domain, Random Reward Domain, and Key Navigation Domain. These environments are characterized by static layouts and logical task structures, requiring the agent to perform operations such as item collection, symbolic reasoning, and sequential decision-making to obtain rewards. The second category includes the Dynamic Obstacles Navigation Domain, which features a grid world with moving obstacles and partial observability. This domain evaluates the adaptability of the framework to dynamic and non-stationary conditions, where agents must learn real-time navigation strategies under uncertainty.

- **Cookie Domain:** The environment consists of three rooms connected by a corridor. Pressing a button in the initial room triggers the stochastic generation of cookies in the other two rooms. The agent receives a reward upon collecting these cookies. This

setting models an “until-type” task with delayed and probabilistic rewards, analogous to inventory replenishment problems where agents must initiate stochastic events and optimize decisions over long horizons. The framework leverages structured reward modeling to encode this stochasticity and temporal decision structure.

- **Keys Domain:** In this environment, the agent navigates through three rooms to reach coffee located in the orange room, earning a reward of +1. Accessing the coffee requires unlocking two brown doors using keys randomly placed at the start of each episode. The agent must infer and track the key locations under partial observability. This domain models sequential access planning under uncertainty, similar to automated tool retrieval systems. The agent is required to execute a temporally extended plan while lacking full state information. The framework captures these temporal dependencies through structured state transitions in the reward automaton.
- **Symbol Domain:** The environment contains blue and green rooms displaying symbols (♣, ♠, ♦), while an orange room may contain a directional arrow and the target symbol. The agent must navigate to the correct symbol room based on this information, receiving +1 for correct matches and −1 for incorrect ones. All other actions yield zero reward. This environment emphasizes symbolic reasoning and perceptual grounding, reflecting tasks such as robotic inspection or symbolic task execution. The modular structure of the reward automaton supports the compositionality required for these tasks, enabling interpretable and robust decision-making.
- **Symbol Matching Domain:** The agent begins in an initial room and selects a task-defining symbol. It then visits three other rooms to find matching symbols. The agent is rewarded +1 for each correct match and penalized −1 for each mismatch. This task requires perceptual alignment and symbolic matching, modeling quality inspection or verification processes in real-world applications. The reward automaton enables consistent matching policies by modeling symbolic dependencies between states and decisions.
- **Random Reward Domain:** In a grid world, the agent presses buttons in the initial room that randomly generate reward points in four mission rooms. A reward of +1 is granted upon reaching a reward point. This environment introduces high reward variance and sparse feedback, modeling applications such as search and rescue or environmental exploration. The reward shaping mechanism in the framework addresses these challenges by guiding exploration with informative potential functions.
- **Key Navigation Domain:** The agent perceives only its current room and must activate a series of keys to unlock doors and reach the goal. A reward is provided upon successful navigation. This domain models adaptive navigation under partial observability and non-stationary task transitions. The framework captures sub-task progression and enables real-time plan adjustment, essential for robust execution in uncertain spatial layouts.
- **Dynamic Obstacles Navigation Domain:** The agent operates in a partially observable grid world where obstacles move with fixed patterns. The agent must reach a designated goal while avoiding collisions. Rewards are given for successful navigation, and penalties are incurred for collisions or invalid moves. This domain tests adaptive policy learning in dynamic and non-stationary environments, representative of autonomous vehicle or drone navigation. The agent must continuously update its strategy in response to dynamic obstacle behavior. Our framework supports such adaptation through structured progress tracking and reward abstraction.

5.2. Performance evaluation and comparison

We initially compared the performance of various models in solving instance problems. The objective of this experiment was to evaluate the convergence speed and reward values obtained during convergence as the training scale increased for each model. To this end, we tested each model’s performance in seven different environments. Fig. 4(a) illustrates the cumulative rewards obtained by each method after every 10,000 training steps, comparing them to the optimal strategy. We reported the highest performance observed across five runs for each method in each environment at different time intervals. The results indicate that RA based on reward shaping outperform other methods to varying degrees across these domains, particularly in more complex environments such as the key navigation domain, where performance is notably superior.

In the method comparison, GLS-PRA consistently achieved the highest reward values across all domains, demonstrating significant advantages in convergence performance and stability. The LRM+DDQN method, utilizing both TS and LS sampling strategies, exhibited lower reward values across various domains compared to LRM+DQRM and GLS-PRA. In contrast, the LRM+DQRM method, which also employed TS and LS strategies, showed markedly improved performance in all domains. For instance, as Table 3 shows, in the cookie domain, LRM+DQRM’s TS and LS achieved reward values of 198.60 and 200.00 respectively, while GLS-PRA attained a reward value of 200.80. Overall, GLS-PRA achieved the highest reward values in all tested domains, highlighting the effectiveness of its optimization strategies and parameter adjustments, and further underscoring the method’s broad applicability and superior performance across different fields.

The results indicate that, in terms of learning efficiency across these domains, methods based on RA significantly outperform the pure DQN algorithm. Furthermore, the GLS-PRA method excels in learning RA more effectively and swiftly. However, in environments with more complex randomness and associations, such as those represented by the ‘symbol match’ or ‘random reward’ domains, these methods might converge to suboptimal strategies. During the relearning process, this could potentially lead to more unstable performance.

In terms of cumulative rewards, GLS-PRA consistently exhibits the highest values across all environments; however, its robustness varies. In the ‘keys’ and ‘key navigation’ domains, the method demonstrates a robust upward learning curve. Conversely, in more complex environments significant fluctuations are evident. Although LRM+DDQN fails to learn optimal strategies in complex environments, resulting in reduced rewards, it maintains its stability, suggesting that increased task complexity impacts performance. The LRM+DQRM method closely follows GLS-PRA across all domains but does not achieve the same level of superior performance.

5.3. Heuristic algorithm performance comparison

We compared the performance of various heuristic algorithms in finding the optimal RA. This experiment aimed to assess the effectiveness of heuristic algorithms in solving large-scale instance problems and to evaluate the performance differences among these algorithms.

Fig. 4(b) illustrates the cumulative rewards obtained after every 10,000 training steps using different search strategies, compared to the optimal strategy. Our observations indicate that solving the TRA problem with genetic local search methods results in faster convergence and superior optimal policies.

Table 3 shows that LS generally outperformed TS, although neither achieved the performance level of GLS-PRA. TS was applied in both LRM+DDQN and LRM+DQRM methods, but typically yielded slightly lower reward values compared to LS. For instance, in the cookie domain, TS achieved reward values of 191.40 and 198.60, whereas LS achieved 196.60 and 200.00, respectively. Overall, LS outperformed TS across various domains; for example, in the keys domain, LS achieved

Table 3
Reward values obtained in convergence rounds in six different domains.

	LRM+DDQN		LRM+DQRM		GLS-PRA
	TS	LS	TS	LS	
cookie	191.40(±0.45)	196.60(±1.48)	198.60 (±1.52)	200.0(±1.22)	200.80(±0.89)
keys	92.20 (±5.15)	89.40 (±4.64)	107.00 (±5.86)	121.40 (±3.13)	132.4(±1.48)
symbol	456.20 (±4.90)	458.20 (±3.29)	468.80 (±3.16)	470.60 (±2.83)	475.2(±2.55)
symbol match	52.42(±10.92)	57.60(±6.88)	51.23(±5.63)	225.21 (±7.27)	320.00(±6.72)
random reward	56.26(±6.92)	66.64(±4.51)	66.22 (±3.13)	81.64 (±6.89)	101.01(±5.36)
key navigation	86.81 (±7.43)	87.27(±9.53)	103.80 (±10.52)	125.8 (±1.52)	133.22(±1.22)

reward values of 89.40 and 121.40, while TS achieved 92.20 and 107.00. However, GLS-PRA, as an independent optimization method, did not rely on TS and LS sampling strategies and consistently showed optimal performance in all tested domains. For instance, in the key navigation domain, GLS-PRA achieved a reward value of 133.22, significantly higher than any combination of LRM+DDQN and LRM+DQRM. This suggests that the superior performance of GLS-PRA across various domains may be attributed to its more effective optimization strategies and parameter adjustment capabilities.

In more complex environments with stronger stochastic correlations, our algorithm demonstrated a more pronounced advantage. This can be attributed to the fact that Tabu Search and Local Search may find solutions that are inferior to those found by Genetic Local Search. In contrast, GLS-PRA leverages the learned RA to effectively guide the agent's learning process towards better expectations, thereby reducing the detrimental impact of suboptimal RA on performance.

We conduct a comprehensive analysis of the computational complexity of our proposed method by evaluating its total runtime — including both training and evaluation phases — against established baseline approaches. Table 4 presents the average runtimes measured across multiple benchmark environments. In both the Random Reward and Key Navigation domains, the proposed GLS-PRA method demonstrates substantial computational efficiency. Specifically, GLS-PRA reduces training time by approximately 28.9% compared to LRM+DDQN and by 17.0% compared to LRM+DQRM in the Random Reward domain. In the Key Navigation domain, the reductions are around 33.9% and 22.1%, respectively.

These efficiency gains stem primarily from the integration of guided learning signals, which facilitate faster policy updates and minimize redundant exploration. Although the GLS-PRA framework introduces additional computational overhead due to potential function approximations and reward reconfiguration, this cost is offset by accelerated convergence, leading to an overall reduction in training time. As a result, GLS-PRA presents a favorable trade-off: a slight increase in computational load yields significant performance improvements, especially in environments with complex dynamics.

We evaluate the framework across six benchmark environments exhibiting diverse levels of stochasticity and partial observability. Domains such as Random Reward and Symbol Match involve high variance in rewards and substantial observation uncertainty, posing considerable challenges for stable policy learning. In contrast, environments like Cookie and Keys are characterized by more deterministic transitions and moderate uncertainty, enabling faster convergence when algorithms are appropriately configured.

Empirical results consistently show that GLS-PRA outperforms all baselines, achieving higher cumulative rewards and faster convergence across all domains. In highly stochastic environments, particularly Symbol Match and Random Reward, GLS-PRA maintains elevated performance despite reward fluctuations, indicating its sensitivity to environmental randomness. By contrast, LRM+DDQN demonstrates lower average rewards but exhibits smoother training curves, reflecting a more conservative exploration-exploitation balance that confers greater robustness under stochastic conditions.

To further evaluate the generalization capacity of our approach, we introduce the Dynamic Obstacles Navigation domain. This domain captures time-dependent complexity and dynamic transitions by featuring

Table 4
Average runtime of the complete training and learning cycle of various methods (minutes).

	LRM+DDQN		LRM+DQRM		GLS-PRA
	TS	LS	TS	LS	
Random Reward Domain	549.89	580.11	471.24	442.30	391.07
Key Navigation Domain	714.38	613.24	583.31	530.71	471.88

a grid world where obstacles move randomly at each time step. The agent must adapt its policy in real time to avoid collisions while reaching a designated target. This domain reflects realistic applications such as autonomous ground or aerial navigation in dynamic environments, placing demands on exploration efficiency and the agent's ability to generalize reward structures under non-stationary, partially observable conditions.

Experimental outcomes in this domain, shown in Fig. 5, indicate that GLS-PRA achieves the highest overall reward and exhibits robust performance across episodes. Moreover, QRM-based methods consistently outperform those using LRM-DQN, highlighting the utility of probabilistic reward abstraction in dynamic and uncertain environments. Among the optimization strategies evaluated, GLS-based variants show the best performance, outperforming both LS and TS methods. LS also outperforms TS, emphasizing the importance of global search and adaptive mutation mechanisms in optimizing reward automata.

These findings reinforce the applicability of GLS-PRA to environments with temporal dynamics and evolving agent–environment interactions. Its consistent performance in the Dynamic Obstacles Navigation domain suggests that GLS-PRA is well suited for real-world applications requiring adaptive planning, online decision-making, and robust generalization under uncertainty and partial observability.

5.4. Impact of reward shaping on learning performance

We investigated the effectiveness of learning RA and examined the practical impact of reward shaping on agent performance. To compute the potential function required for shaping, we adopted a value iteration strategy. This method relies on dynamic programming to iteratively update value estimates until convergence. Despite its computational cost, value iteration yields highly accurate potential estimates, making it well-suited for environments with intricate state transitions and complex reward dependencies.

Fig. 6 illustrates the distribution of cumulative rewards under different reward shaping strategies when solving the TRA problem using GLS-PRA. We compare the outcomes of applying reward shaping against those obtained without it. Among the strategies evaluated, the value iteration-based shaping consistently achieves superior results, guiding the agent more efficiently toward goal states by leveraging informative potential-based signals.

Further analysis of convergence behavior, as reported in Table 5, reveals that reward shaping leads to notable improvements in both the magnitude and consistency of cumulative rewards across multiple environments. The value iteration strategy outperformed linear and

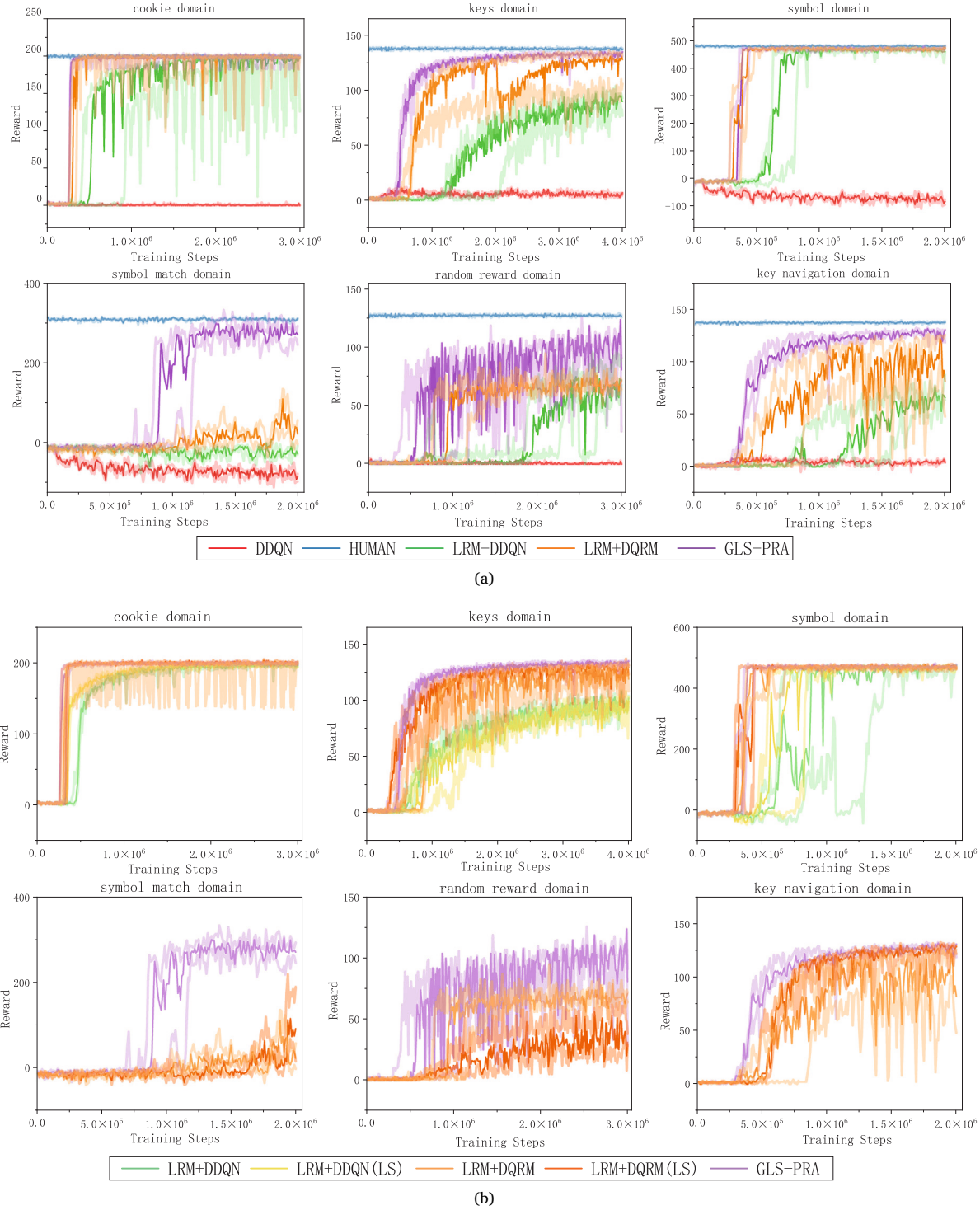


Fig. 4. Performance comparison of different methods and search strategies based on cumulative rewards over training time. Subfigure (a) reports the cumulative rewards obtained by each method at every 10,000 training steps, benchmarked against the optimal strategy. GLS-PRA demonstrate consistently superior performance, particularly in challenging environments such as the key navigation domain. Subfigure (b) shows the impact of various search strategies on cumulative rewards, where genetic local search achieves both faster convergence and higher policy quality compared to alternative strategies.

exponential shaping methods, which, while capable of boosting reward levels, introduced noticeable instability during training.

In the Symbol Match Domain, reward shaping increased the final reward from 227.80 to 320.00. In the Random Reward and Key Navigation domains, rewards improved from 83.62 to 101.01 and from 95.40 to 133.22, respectively. These quantitative results highlight the

effectiveness of potential-based shaping in accelerating convergence and improving policy quality, especially in environments with high stochasticity or complex sequential dependencies.

Moreover, reward shaping contributed to a significant reduction in the variability of reward values, enhancing training stability. For instance, in the Key Navigation domain, the standard deviation decreased

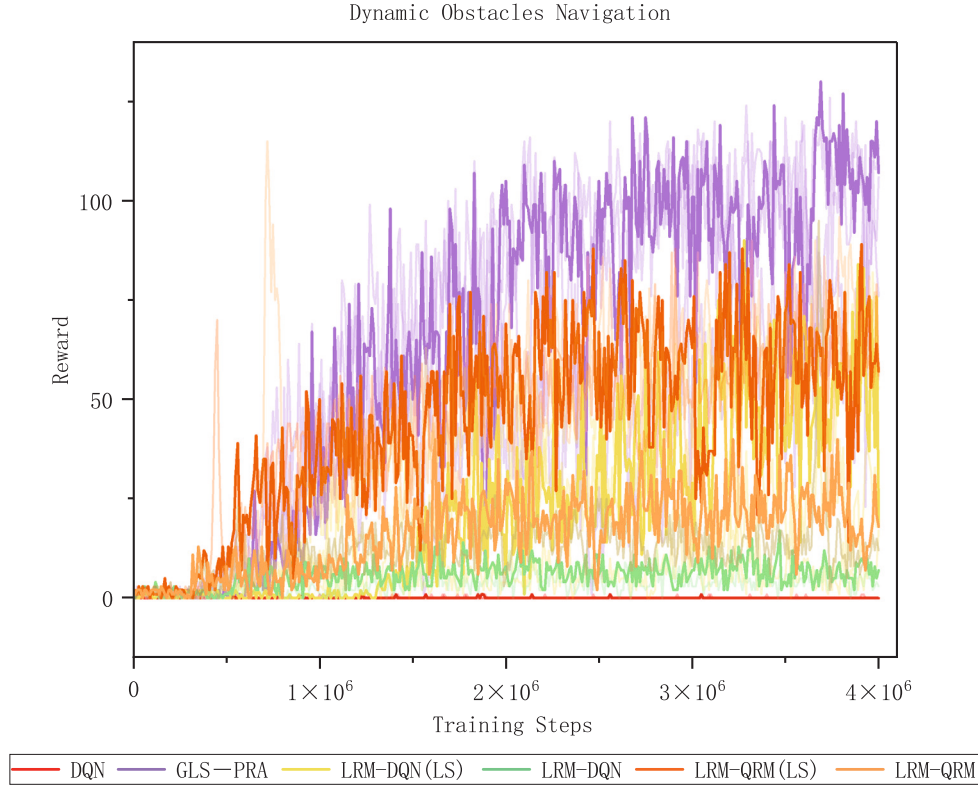


Fig. 5. Cumulative rewards achieved by each method every 10,000 training steps in the Dynamic Obstacles Navigation domain, compared to the optimal strategy. The results show that GLS-PRA achieves the best performance.

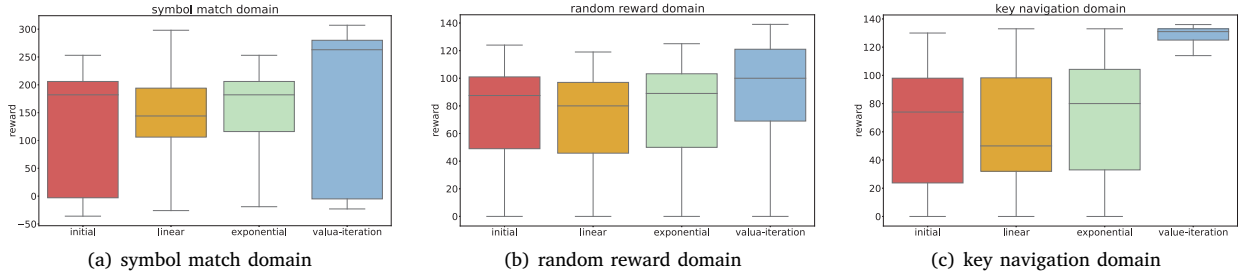


Fig. 6. Box plots of reward values obtained with different strategies in various domains.

from 6.01 to 1.22, and in the Symbol Match domain, from 23.20 to 6.72. This reduction in variance indicates that the learning process became more robust and consistent, further validating the reliability of reward shaping in uncertain environments.

In summary, reward shaping not only boosts the average reward level but also substantially enhances learning stability. These benefits are especially evident in the Symbol Match and Key Navigation domains, confirming the practical effectiveness of shaping techniques in improving policy convergence. By exploiting accurately computed potential functions, agents receive better guidance toward optimal action sequences, thereby reducing the adverse effects of environmental noise. The observed reduction in reward variance across domains evidences the stabilizing impact of shaping. However, existing methods remain vulnerable in highly stochastic and partially observable environments, where minor perturbations in reward structures can still impede convergence. This underscores the need for future approaches that explicitly incorporate environmental uncertainty into the shaping mechanism.

5.5. Hyper-parameter analysis

Lastly, we explored the impact of balancing exploration and exploitation on the learning performance of the method. In reinforcement learning, the parameter ϵ plays a crucial role in managing this balance, significantly affecting the quality and speed of the learning process. Due to varying demands for exploration and exploitation, the optimal ϵ value differs across environments. Figs. 7(a), 7(b), and 7(c) present box plots of the cumulative rewards obtained every 10,000 training steps using the GLS-PRA method with different ϵ values in solving the TRA problem. In the key navigation environment, higher ϵ values result in better performance but slower learning and convergence speeds. Conversely, in the symbol match environment, lower ϵ values lead to faster learning and earlier convergence, although stability is relatively poorer.

Our observations indicate that excessively high ϵ values can cause the agent's behavior to be overly random, making it difficult to consistently learn effective strategies. Conversely, excessively low ϵ values

Table 5
Impact of reward shaping on different strategies.

Domain	No reward shaping	Linear strategy	Exponential strategy	Value iteration strategy
symbol match	227.80(± 23.2)	243.89(± 7.09)	268.95(± 17.05)	320.00(± 6.72)
random reward	83.62(± 29.33)	83.72(± 23.22)	87.59(± 13.90)	101.01(± 5.36)
key navigation	95.40(± 6.01)	96.45(± 6.76)	97.81(± 9.27)	133.22(± 1.22)

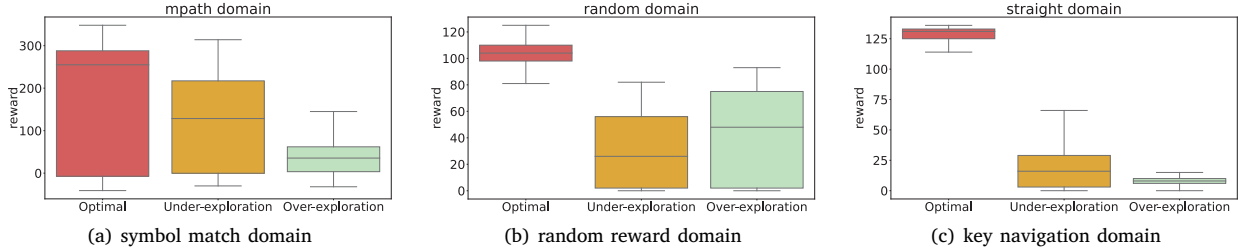


Fig. 7. Box plots of reward values obtained with different epsilon intervals in various domains.

may prevent the agent from adequately exploring, thus limiting learning performance. However, within a certain range, the impact of ϵ values on the experimental results is limited, though there is a tendency to converge towards the optimal value. To determine the optimal ϵ value, we reference reward values: if there is no significant improvement in rewards over a period, increasing the ϵ value can enhance exploration. Conversely, if rewards continue to improve, decreasing the ϵ value can enhance exploitation.

Building on the preceding empirical findings, we introduce a dynamic ϵ -adjustment strategy designed to adapt exploration behavior to the complexity and stochasticity of the environment. In highly stochastic and partially observable domains, such as the *Random Reward* and *Symbol Match* domains, exploration must be sustained for longer durations to enable effective policy adaptation under volatile dynamics. Accordingly, a slower decay of ϵ is warranted. In contrast, in more deterministic settings — such as the *Cookie* and *Keys* domains — rapid convergence is advantageous, and thus a faster ϵ decay promotes early exploitation once sufficient exploration is achieved.

To formalize this approach, we define the dynamic update rule in Eq. (22):

$$\epsilon_{t+1} = \begin{cases} \min(\epsilon_{\max}, \epsilon_t + \Delta\epsilon) & \text{if } \Delta R_t < \theta, \\ \max(\epsilon_{\min}, \epsilon_t \times \gamma) & \text{if } \Delta R_t \geq \theta, \end{cases} \quad (22)$$

where ΔR_t denotes the change in cumulative reward over a recent evaluation window, θ is a predefined reward improvement threshold, $\gamma \in (0, 1)$ is a multiplicative decay factor, and $\Delta\epsilon > 0$ is an additive increment encouraging additional exploration when progress is insufficient. The parameters $\epsilon_{\max}$ and $\epsilon_{\min}$ define the permissible range of exploration probabilities.

This adaptive scheduling mechanism allows the agent to balance exploration and exploitation in response to real-time learning signals, rather than adhering to a fixed decay schedule. Such flexibility is especially valuable in dynamic or non-stationary environments where the level of uncertainty or complexity may vary during training. By responding to reward trends, the proposed strategy enhances learning robustness and facilitates more efficient policy development across diverse environmental conditions.

5.6. Threats to validity

The learning curves observed in the *Dynamic Obstacles Navigation* domain, as shown in Fig. 5, exhibit significant fluctuations. This variability is primarily attributed to the stochastic nature of obstacle behavior, including both their probabilistic appearance and spatial positioning. The dynamic placement of obstacles at varying time steps

introduces pronounced non-determinism, resulting in a continuously shifting environment that complicates the agent's learning process. Consequently, the optimal trajectory to the goal can vary substantially between episodes, leading to inconsistent reward sequences and impeding stable policy convergence. These fluctuations increase the learning difficulty and contribute to high variance in performance across different runs.

Despite demonstrating empirical effectiveness, the current implementation of GLS-PRA exhibits limitations that restrict its applicability in more complex scenarios. In particular, its performance may deteriorate in environments characterized by high stochasticity or partial observability. This degradation arises from the sensitivity of the learned reward automaton to noisy transitions and ambiguous observational cues. Although the framework employs dynamic reward shaping to mitigate instability, it lacks inherent mechanisms for uncertainty modeling or quantification. As a result, GLS-PRA is susceptible to overfitting and may struggle to converge reliably in volatile settings.

6. Conclusions and future work

In this work, we propose a framework for learning reward mechanisms in partially observable environments, addressing challenges that standard deep reinforcement learning approaches often fail to overcome. Drawing from the theory of POMDPs and RAs, we define the key properties required for RA-based policy synthesis under partial observability. By casting RA learning as a discrete optimization problem, we develop an efficient reward shaping method that improves learning stability and convergence. Our method integrates RA construction with policy learning, resulting in higher cumulative rewards and lower variance across diverse environments. Among the evaluated optimization strategies, genetic local search consistently achieves the best performance in terms of both reward maximization and learning stability.

The proposed GLS-PRA framework demonstrates strong empirical results in discrete-state, single-agent, partially observable environments. It is readily extensible to more complex settings, including multi-agent and multi-objective reinforcement learning. For multi-agent systems, individual or shared reward automata can model inter-agent dependencies, enabling coordinated behavior under dynamic joint policies. In multi-objective tasks, the decomposition of reward signals into modular automata provides a principled way to handle trade-offs, improve interpretability, and incorporate external constraints. These structural advantages position GLS-PRA as a promising tool for scalable and explainable reinforcement learning in domains such as autonomous robotics, logistics, and adaptive control.

Future work will focus on enhancing the robustness and scalability of GLS-PRA through three main directions. First, we aim to incorporate distributional reinforcement learning and ensemble-based uncertainty modeling to improve resilience in stochastic environments. Second, we plan to develop lightweight approximations for genetic search and potential-based reward shaping to reduce computational overhead. Third, we intend to extend the framework to continuous domains via function approximation and modular reward representations. These enhancements will further strengthen GLS-PRA's capability in handling uncertainty, complexity, and large-scale decision-making.

CRedit authorship contribution statement

Zhengwei Zhu: Writing – original draft, Methodology, Conceptualization. **Zhixuan Chen:** Writing – original draft, Software, Data curation. **Chenyang Zhu:** Writing – review & editing, Visualization, Validation, Supervision, Funding acquisition. **Wen Si:** Writing – review & editing, Visualization, Validation, Formal analysis. **Fang Wang:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- Anandkumar, A., Ge, R., Hsu, D., Kakade, S.M., Telgarsky, M., 2012. Tensor Decompositions for Learning Latent Variable Models. Tech. Rep., 15, (1), Defense Technical Information Center, pp. 2773–2832. <http://dx.doi.org/10.21236/ada604494>.
- Bourel, H., Jonsson, A., Maillard, O.-A., Talebi, M.S., 2023. Exploration in reward machines with low regret. In: *Proceedings of Machine Learning Research (PMLR)*, AISTATS, vol. 206, pp. 4114–4146.
- Camacho, A., Varley, J., Zeng, A., Jain, D., Iscen, A., Kalashnikov, D., 2021. Reward machines for vision-based robotic manipulation. In: 2021 IEEE International Conference on Robotics and Automation. ICRA, IEEE, IEEE, pp. 14284–14290. <http://dx.doi.org/10.1109/icra48506.2021.9561927>.
- Curtis, A., Kaelbling, L., Jain, S., 2023. Task-directed exploration in continuous pomdps for robotic manipulation of articulated objects. In: 2023 IEEE International Conference on Robotics and Automation. ICRA, IEEE, pp. 3721–3728.
- Dohmen, T., Topper, N., Atia, G., Beckus, A., Trivedi, A., Velasquez, A., 2022. Inferring probabilistic reward machines from non-Markovian reward signals for reinforcement learning. *Proc. Int. Conf. Autom. Plan. Sched.* 32, 574–582. <http://dx.doi.org/10.1609/icaps.v32i1.19844>.
- Furelos-Blanco, D., Law, M., Jonsson, A., Broda, K., Russo, A., 2021. Induction and exploitation of subgoal automata for reinforcement learning. *J. Artificial Intelligence Res.* 70, 1031–1116. <http://dx.doi.org/10.1613/jair.1.12372>.
- Gangwani, T., Lehman, J., Liu, Q., Peng, J., 2020. Learning belief representations for imitation learning in pomdps. In: *Uncertainty in Artificial Intelligence*. PMLR, pp. 1061–1071.
- Golowich, N., Moitra, A., Rohatgi, D., 2022. Planning in observable POMDPs in quasipolynomial time. *arXiv preprint arXiv:2201.04735*. URL <http://arxiv.org/abs/2201.04735v2>.
- Hasanbeig, M., Abate, A., Kroening, D., 2018. Logically-constrained reinforcement learning. *arXiv preprint arXiv:1801.08099*. URL <http://arxiv.org/abs/1801.08099v8>.
- Icarte, R.T., Klassen, T.Q., Valenzano, R.A., McIlraith, S.A., 2018. Teaching multiple tasks to an RL agent using ltl. In: *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*. pp. 452–461, URL <http://dl.acm.org/citation.cfm?id=3237452>.
- Icarte, R.T., Waldie, E., Klassen, T.Q., Valenzano, R.A., Castro, M.P., McIlraith, S.A., 2019. Learning Reward Machines for Partially Observable Reinforcement Learning., vol. 32, pp. 15497–15508, URL <https://proceedings.neurips.cc/paper/2019/hash/532435c44bec236b471a47a88d63513d-Abstract.html>.
- Igl, M., Zintgraf, L., Le, T.A., Wood, F., Whiteson, S., 2018. Deep variational reinforcement learning for POMDPs. In: *International Conference on Machine Learning*. PMLR, pp. 2117–2126.
- Illanes, L., Yan, X., Toro Icarte, R., McIlraith, S.A., 2020. Symbolic plans as high-level instructions for reinforcement learning. *Proc. Int. Conf. Autom. Plan. Sched.* 30, 540–550. <http://dx.doi.org/10.1609/icaps.v30i1.6750>.
- Jahromi, M.J., Jain, R., Nayyar, A., 2022. Online learning for unknown partially observable mdps. In: *International Conference on Artificial Intelligence and Statistics*. PMLR, pp. 1712–1732.
- Jin, C., Kakade, S., Krishnamurthy, A., Liu, Q., 2020. Sample-efficient reinforcement learning of undercomplete pomdps. *Adv. Neural Inf. Process. Syst.* 33, 18530–18539.
- Jin, C., Yang, Z., Wang, Z., Jordan, M.I., 2023. Provably efficient reinforcement learning with linear function approximation. *Math. Oper. Res.* 48 (3), 1496–1521. <http://dx.doi.org/10.1287/moor.2022.1309>.
- Lacerda, B., Parker, D., Hawes, N., 2014. Optimal and dynamic planning for Markov decision processes with co-safe LTL specifications. In: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems. 15, IEEE, CiteSeer, pp. 1587–1593. <http://dx.doi.org/10.1109/iroso.2014.6942756>.
- Laurent, P.A., 2008. The emergence of saliency and novelty responses from reinforcement learning principles. *Neural Netw.* 21 (10), 1493–1499. <http://dx.doi.org/10.1016/j.neunet.2008.09.004>.
- Lauri, M., Hsu, D., Pajarinen, J., 2022. Partially observable markov decision processes in robotics: A survey. *IEEE Trans. Robot.* 39 (1), 21–40.
- Levin, K., Pappas, N., Karapantelakis, A., Feljan, A., Seipp, J., 2024. Numeric reward machines. In: *International Conference on Machine Learning*. ICML.
- Levinson, J., Askeland, J., Becker, J., Dolson, J., Held, D., Kammel, S., Kolter, J.Z., Langer, D., Pink, O., Pratt, V., Sokolsky, M., Stanek, G., Stavens, D., Teichman, A., Werling, M., Thrun, S., 2011. Towards fully autonomous driving: Systems and algorithms. In: 2011 IEEE Intelligent Vehicles Symposium. IV, IEEE, IEEE, pp. 163–168. <http://dx.doi.org/10.1109/ivs.2011.5940562>, URL <https://doi.org/10.1109/ivs.2011.5940562>.
- Lu, C., Shi, R., Liu, Y., Hu, K., Du, S.S., Xu, H., 2024. Rethinking transformers in solving partially observable Markov decision processes. *arXiv preprint arXiv:2405.17358*.
- Morad, S., Kortvelesy, R., Bettini, M., Liwicki, S., Prorok, A., 2023. Popgym: Benchmarking partially observable reinforcement learning. *arXiv preprint arXiv:2303.01859*. URL <http://arxiv.org/abs/2303.01859v1>.
- Mossel, E., Roch, S., 2005. Learning nonsingular phylogenies and hidden Markov models. In: *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*. ACM, pp. 366–375. <http://dx.doi.org/10.1145/1060590.1060645>.
- Neary, C., Xu, Z., Wu, B., Topcu, U., 2020. Reward machines for cooperative multi-agent reinforcement learning. *arXiv preprint arXiv:2007.01962*. URL <http://arxiv.org/abs/2007.01962v2>.
- Papadimitriou, C.H., Tsitsiklis, J.N., 1987. The complexity of Markov decision processes. *Math. Oper. Res.* 12 (3), 441–450. <http://dx.doi.org/10.1287/moor.12.3.441>.
- Paracá, R., Nodari, L., Ardon, L., Furelos-Blanco, D., Cerutti, F., Russo, A., 2024. Learning robust reward machines from noisy labels. In: *Principles of Knowledge Representation and Reasoning*. KR.
- Pleines, M., Pallasch, M., Zimmer, F., Preuss, M., 2023. MemoryGym: Partially observable challenges to memory-based agents. In: *International Conference on Learning Representations*. ICLR.
- Toro Icarte, R., Klassen, T.Q., Valenzano, R., Castro, M.P., Waldie, E., McIlraith, S.A., 2023. Learning reward machines: A study in partially observable reinforcement learning. *Artificial Intelligence* 323, 103989. <http://dx.doi.org/10.1016/j.artint.2023.103989>.
- Toro Icarte, R., Klassen, T.Q., Valenzano, R., McIlraith, S.A., 2022. Reward machines: Exploiting reward function structure in reinforcement learning. *J. Artificial Intelligence Res.* 73, 173–208. <http://dx.doi.org/10.1613/jair.1.12440>.
- Toro-Rey, R., Perez, M., Somenzi, F., 2023. Contextual pre-planning on reward machine abstractions for deep reinforcement learning. *arXiv:arXiv:2307.05209*.
- Valiente, R., Toghi, B., Pedarsani, R., Fallah, Y.P., 2022. Robustness and adaptability of reinforcement learning-based cooperative autonomous driving in mixed-autonomy traffic. *IEEE Open J. Intell. Transp. Syst.* 3, 397–410.
- Vlassis, N., Littman, M.L., Barber, D., 2012. On the computational complexity of stochastic controller optimization in POMDPs. *ACM Trans. Comput. Theory* 4 (4), 1–8. <http://dx.doi.org/10.1145/2382559.2382563>.
- Zanette, A., Lazaric, A., Kochenderfer, M., Brunskill, E., 2020. Learning near optimal policies with low inherent bellman error. In: *International Conference on Machine Learning*. PMLR, pp. 10978–10989.
- Zheng, X., Yu, C., Zhang, M., 2022. Lifelong reinforcement learning with temporal logic formulas and reward machines. *Knowl.-Based Syst.* 257, 109650. <http://dx.doi.org/10.1016/j.knsys.2022.109650>.
- Zhu, C., Cai, Y., Hu, C., Bi, J., 2022. Efficient reinforcement learning with generalized-reactivity specifications. In: 2022 29th Asia-Pacific Software Engineering Conference. APSEC, IEEE, IEEE, pp. 31–40. <http://dx.doi.org/10.1109/apsec57359.2022.00015>.
- Zhu, C., Zhu, J., Si, W., Wang, X., Wang, F., 2024. Multi-agent reinforcement learning with synchronized and decomposed reward automaton synthesized from reactive temporal logic. *Knowl.-Based Syst.* 306, 112703.